

Product Requirements Document (PRD)

Secure Hybrid Data Center Reference Architecture – Simulation & IaC Build

Built using Google Antigravity (agentic development platform)

Project	Cisco Virtual Internship 2026 – Cyber Security Track
Deliverable	Working, testable Infrastructure-as-Code (IaC) + policy simulation of the proposed Zero-Trust Hybrid Data Center architecture
Build tool	Google Antigravity (agent-first IDE/platform)
Document owner	Student / Intern
Status	Draft v1.0

1. Background

The source design document (Procedure-1 & Procedure-2) specifies a Zero-Trust, defense-in-depth architecture connecting a private data center to public cloud, spanning IAM, network segmentation, Kubernetes micro-segmentation, hybrid connectivity, and centralized monitoring. That document is an **architecture proposal**, not a build artifact. This PRD defines what needs to be *built* — as code, policy, and simulated infrastructure — to demonstrate the architecture works, and how **Google Antigravity's agent-first workflow** will be used to build it.

Core principle carried over from the design doc: **a compromised application must not automatically become a compromised enterprise**. Every requirement below is written to make that principle testable, not just described.

2. Problem Statement

The design document answers *what* the architecture should look like. It does not yet produce:

- Deployable, version-controlled infrastructure definitions (VPC/VNet, subnets, security groups, transit/hub topology)
- Enforceable IAM policy artifacts (roles, permission boundaries, federation config)

- Kubernetes manifests implementing namespaces, RBAC, and NetworkPolicies
- A repeatable way to *prove* blast-radius containment (the Section 14 attack scenario) rather than just describe it
- Monitoring/logging configuration tying the above together

This project closes that gap by producing a working simulated environment plus the automation/policy code behind it.

3. Goals & Success Criteria

Goal	Success Metric
Codify the Hub-and-Spoke hybrid network	IaC (Terraform) provisions 1 Hub VPC + ≥ 3 Spoke VPCs with no transitive routing by default
Enforce default-deny segmentation	Every security group / NSG and every K8s NetworkPolicy starts default-deny; only explicitly listed flows pass
Demonstrate blast-radius containment	The Section 14 attack scenario (compromised research app) is re-run against the built environment and the same containment result is verified with

Goal	Success Metric
	evidence (logs/test output), not just asserted
Implement least-privilege IAM	IAM roles/service accounts map 1:1 to workloads described in Section 6/10 tables; no wildcard * permissions
Centralize visibility	All environments ship logs to one collector/log sink; a query can show cross-zone denied traffic
Reproducibility	A fresh reviewer can <code>git clone</code> + run one setup command and reach a working demo environment

4. Scope

In scope

- IaC for the Hub/Spoke VPC topology, subnets, routing, security groups (cloud-agnostic design, single cloud provider implementation – see Assumptions)
- Kubernetes manifests: namespaces, RBAC, NetworkPolicies, Pod Security Standards, for the 4 workloads in Section 10 (Student-App, Faculty-App, Exam-App, Research-App)

- IAM/identity artifacts: role definitions, service-account-to-role mappings, permission boundaries, sample SSO/MFA policy config (as policy-as-code, not a live IdP deployment)
- Firewall/security-group rule sets matching the Section 9 table exactly
- A logging/monitoring stack (or simulated equivalent) capturing security-group denies, K8s audit events, and auth events into one place
- Attack-scenario test harness reproducing Section 14
- Documentation: architecture diagrams generated from code, README, runbook

Out of scope

- Procuring or configuring real Cisco hardware/licenses (ISE, Secure Firewall, Secure Workload) – represented as **stubs/mocks with a clear technology-mapping note**, per the source document's own guidance that Cisco tooling is optional and infra-dependent
- Production faculty/student onboarding or real user data
- A real dedicated connection (Direct Connect/ExpressRoute) – simulated via IPsec VPN or local network emulation only
- Cost optimization / production-scale capacity planning

5. Users / Personas

Persona	Need from this build
Reviewing faculty/instructor	Can inspect IaC and see segmentation matches the design doc; can run the attack-scenario test and see containment proven
Fellow student/reviewer	Can clone and stand up the demo in a single environment without manual cloud console clicking
"Security team" role (project self-assessment)	Can query centralized logs to answer "what got blocked and why"

6. Functional Requirements

6.1 Network & Segmentation (maps to Doc §3, §4, §6, §22, §23)

- FR-1: Provision one Hub VPC (shared services/transit/inspection) and ≥ 3 Spoke VPCs (Student, Faculty/Exam, Research)
- FR-2: No default transitive routing between Spoke VPCs; cross-VPC traffic only via an explicit inspected path

- FR-3: Each VPC has public (LB/ingress), private (workload), and data subnets; data subnets have no internet route
- FR-4: Security groups default-deny; rules reference source security groups, not open CIDRs, matching the Doc §9 table exactly (Internet → Web:443 allow, Web → App:8080 allow, App → DB:5432 allow, Internet → DB:deny, Student VPC ↔ Exam VPC:deny, App → K8s API:deny)

6.2 Hybrid Connectivity (maps to Doc §5, §22)

- FR-5: Simulate the private-DC ↔ cloud link via IPsec VPN with a firewall/inspection point on both sides
- FR-6: Only required prefixes are routed across the hybrid link (explicit route tables, not 0.0.0.0/0)

6.3 IAM & Zero Trust (maps to Doc §7, §24)

- FR-7: Define least-privilege IAM roles for: application developer, network engineer, cloud engineer, Kubernetes engineer, security/SOC, admin
- FR-8: Workload identity (no static credentials) maps each K8s service account to one scoped cloud role
- FR-9: Policy-as-code enforces no long-lived access keys and blocks overly broad security-group creation (permission boundary / org-policy equivalent)

6.4 Kubernetes Micro-segmentation (maps to Doc §10, §23)

- FR-10: One namespace per workload (student, faculty, exam, research) with RBAC scoping developers to their own namespace
- FR-11: Default-deny NetworkPolicy per namespace; explicit allow rules matching the Doc §10 allowed/blocked table
- FR-12: Pod Security Standards set to Restricted (or equivalent) on all namespaces

6.5 Monitoring & Attack Simulation (maps to Doc §15, §14, §27)

- FR-13: Central log sink ingests: security-group deny events, K8s audit logs, auth events
- FR-14: An automated test/script reproduces the Section 14 scenario — simulate a compromised research-app pod/credential — and asserts: research DB reachable, Student-App/Exam-App/Admin/K8s-control-plane unreachable
- FR-15: Test output (pass/fail + evidence) is captured as a build artifact (report), not just console output

7. Non-Functional Requirements

- NFR-1: All infrastructure defined as code, version-controlled, no manual console configuration for

anything in scope

- NFR-2: Idempotent deploy – re-running the IaC produces no drift/errors
- NFR-3: Full environment stands up and tears down with single commands (`apply` / `destroy`)
- NFR-4: Secrets never committed to source; pulled from a secrets manager or local `.env` excluded via `.gitignore`
- NFR-5: Documentation and diagrams stay in sync with code (diagrams generated from or validated against the IaC where feasible)

8. Build Plan Using Google Antigravity

Google Antigravity is an agent-first IDE/platform (VS Code-based) where you assign **task-level objectives** to autonomous agents that plan, implement, test, and produce verifiable **Artifacts** (task plans, screenshots, browser recordings, walkthroughs) rather than just chat replies. The build should be organized as a sequence of Antigravity agent tasks, one per phase, each producing its own verifiable Artifact – mirroring the Doc §11 / §29 deployment workflow.

Phase	Antigravity Agent Task	Expected Artifact
1. Repo & environment	"Set up a Terraform +	Repo scaffold + README task

Phase	Antigravity Agent Task	Expected Artifact
scaffolding	Kubernetes manifest repo structure for a hub-and-spoke hybrid architecture with folders for network, iam, k8s, monitoring, tests"	plan
2. Network IaC	"Implement Hub VPC + 3 Spoke VPCs, subnets, and security groups per FR-1–FR-4" (feed in the Doc §9 table directly as the spec)	Terraform plan/apply output, diagram
3. Hybrid connectivity	"Add IPsec VPN + inspection point between simulated on-prem and Hub VPC per FR-5–FR-6"	Terraform diff, route table screenshot

Phase	Antigravity Agent Task	Expected Artifact
4. IAM policy-as-code	"Generate least-privilege IAM roles and workload identity bindings per FR-7–FR-9"	Policy files + validation test results
5. Kubernetes manifests	"Create namespaces, RBAC, NetworkPolicies, Pod Security Standards per FR-10–FR-12, matching the allowed/blocked table"	<code>kubectl</code> apply output, policy diagrams
6. Logging/monitoring	"Wire security-group denies, K8s audit logs, and auth events into one log sink per FR-13"	Dashboard/query screenshot
7. Attack-scenario test harness	"Write and run an automated test that simulates a	Test report artifact (pass/fail + evidence)

Phase	Antigravity Agent Task	Expected Artifact
	compromised research-app workload and asserts containment per FR-14–FR-15"	
8. Verification & docs	"Review all prior artifacts against the FR list, flag gaps, regenerate architecture diagram from final code"	Final verification report

Working pattern in Antigravity:

1. Paste the relevant FR block (or the source design doc section) as the task's objective/spec — Antigravity performs best with concrete, verifiable specs rather than vague asks.
2. Let the agent work asynchronously in its own space (editor + terminal + browser) rather than only in a chat sidebar; review its **Artifact** (plan + verification) before merging, not just the diff.
3. Use the Agent Manager to run Phases 2, 4, and 5 as parallel subagent tasks once Phase 1 scaffolding exists, since they touch independent directories.

4. Gate each phase's merge on its test/verification artifact — do not proceed to the next phase until the current phase's Artifact shows a pass.
5. Reserve the final Phase 8 task for cross-checking the whole build against Section 14's attack scenario and the FR table in this PRD, since that's the architecture's core claim.

9. Deliverables Checklist

- ☐ Terraform (or equivalent) repo: Hub + Spoke VPCs, subnets, security groups
- ☐ IPsec/hybrid connectivity config
- ☐ IAM policy-as-code + workload identity bindings
- ☐ Kubernetes manifests: namespaces, RBAC, NetworkPolicies, Pod Security Standards
- ☐ Central logging/monitoring wiring
- ☐ Automated attack-scenario test + report
- ☐ Architecture diagram (generated/kept in sync with code)
- ☐ README + runbook (deploy, test, teardown)
- ☐ Cisco technology-mapping note (Doc §16) explaining what each stub represents in a real deployment

10. Assumptions & Open Questions

- **Assumption:** Build targets a single public cloud provider for the implementation (recommend picking one — AWS/Azure/GCP — even though the source doc is multi-cloud in principle); confirm which before Phase 2.
- **Assumption:** "Private data center" is simulated (e.g., a local network namespace, second VPC, or on-prem emulator) rather than physical hardware.
- **Open question:** Should Cisco-specific tooling (ISE, Secure Firewall) be mocked with equivalent open-source/cloud-native tools (e.g., OPA/Gatekeeper for policy, native cloud firewall for Secure Firewall) — recommended, per Doc §16's own note that Cisco products are optional.
- **Open question:** Scope of "centralized logging" — a real SIEM, or a lightweight stand-in (e.g., cloud-native log service + a query dashboard) is sufficient for demonstrating FR-13–FR-15.

11. Risks

Risk	Mitigation
Agent-generated IaC drifts from the Doc §9 security-group table	Feed the exact table as the spec text for Phase 2's task; verify line-by-line in Phase 8

Risk	Mitigation
Cloud costs from standing up multiple VPCs/clusters	Use smallest viable instance sizes; tear down between work sessions; consider <code>kind / minikube</code> for the K8s layer instead of a managed cluster
Attack-scenario test is superficial (asserts too little)	Require the Phase 7 artifact to show both the allowed path (research DB) and every blocked path (Student, Exam, Admin, K8s control plane), not just one